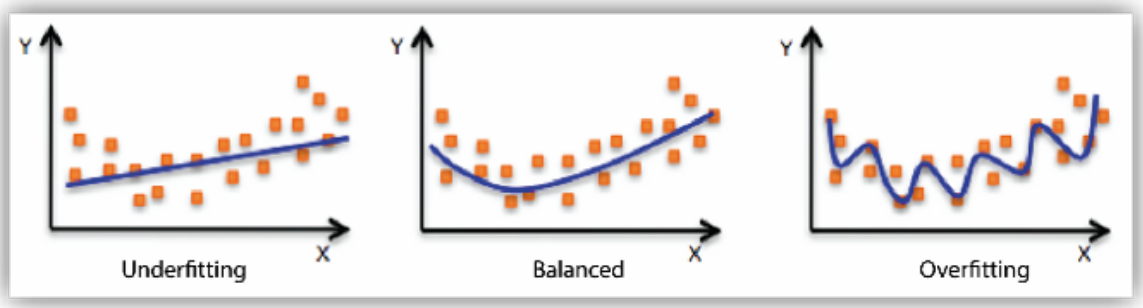
Neural Network Generalization

Generalization is a term used to describe a model's ability to react to new data.

Generalization is the ability of your model, after being trained to digest new data and make accurate predictions. It is probably the most important element of your AI project. A model's ability to generalize is central to the success of an AI project. Indeed, our fear is that a model has been trained too well on training data but is unable to generalize.

We often don't reach production stage because of this… It will make inaccurate predictions when given new data, making the model useless even though it is able to make accurate predictions for the training data. **This is called overfitting.**

The opposite can also happen. ***Underfitting*** *is when a model has not been trained enough on the data. In the case of underfitting, it makes the model just as useless and it is not capable of making accurate predictions, even with the training data.*



When it comes to neural networks, regularization is a technique that makes slight modifications to the learning algorithm such that the model generalizes better. This, in turn, improves the model's performance on the unseen data as well.

Model Complexity

Proper regularization is a critical reason for better generalization performance because deep neural networks are often over-parametrized and likely to suffer from overfitting problems.

We can reduce the complexity of a neural network to reduce overfitting by:

Changing the network structure (number of weights)	Changing the network parameters (values of weights)

Reducing the capacity of your model reduces the likelihood of the model overfitting the training dataset, to a point where it no longer overfits.

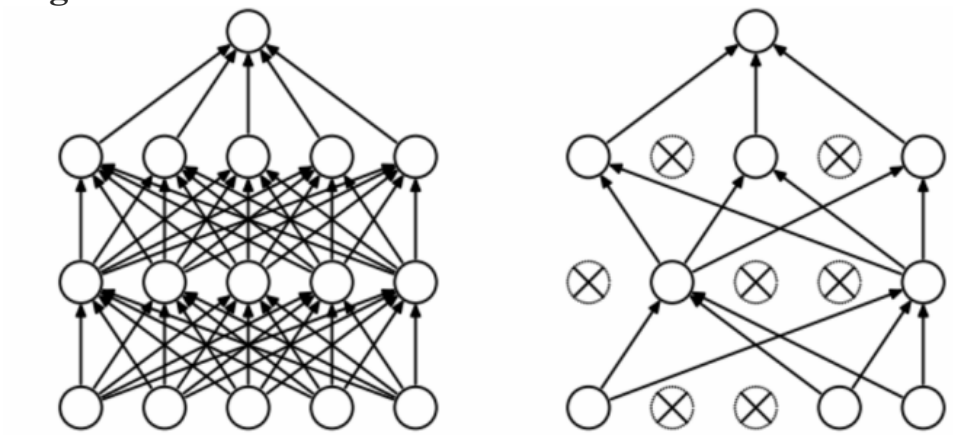
Techniques that seek to reduce overfitting by keeping network weights small are referred to as **regularization methods**.

Regularization: A class of approaches that add additional information to transform an ill-posed problem into a more stable well-posed problem.

Below, listed several regularization methods that we often use (**other methods do exist** such as weight constraints or activity regularization).

Dropout

Based on my experience, this approach is by far the most effective regularization technique. Dropout is applied between two successive layers in a network. At each iteration, the key idea is to randomly drop units (along with their connections) from the neural network. This causes the subsequent layer rely on **all** of its connections to the previous layer. **This approach helps fight overfitting by preventing complex co-adaptations on training data.**



(a) Standard Neural Net (b) After applying dropout.

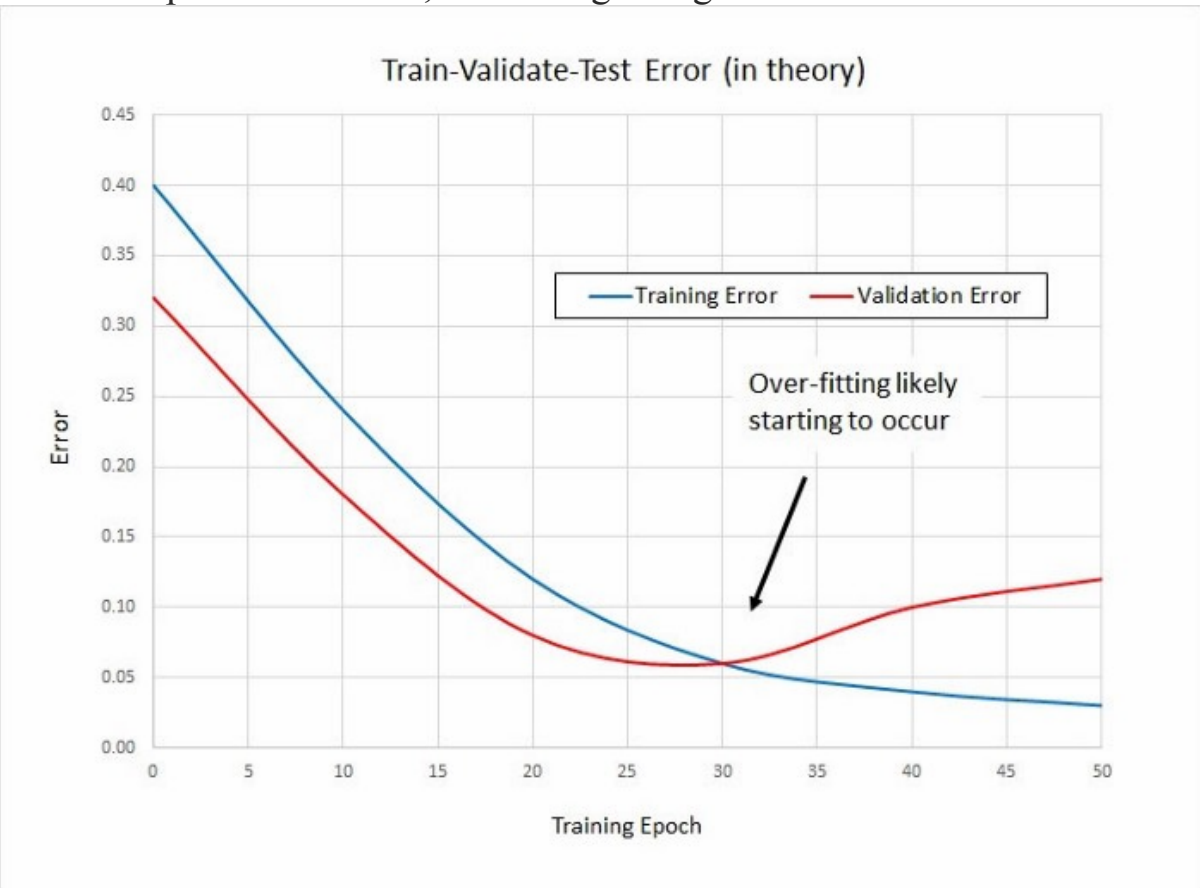
Noise

A common type of regularization is to inject noises during training: adding or multiplying noise to hidden units of the neural networks. By allowing some inaccuracy when training deep neural networks, not only the training performance but also the accuracy of the model can be improved.

Early Stopping

Early stopping is a kind of cross-validation strategy where we keep one part of the training set as the validation set. In reality, when we see that the performance on the validation set is getting worse, we stop the training on the model.

In other words, this approach attempts to stop an estimator's training phase early, at the point where it has learned to extract all meaningful relationships from the data, before beginning to model its noise.



Transfer learning

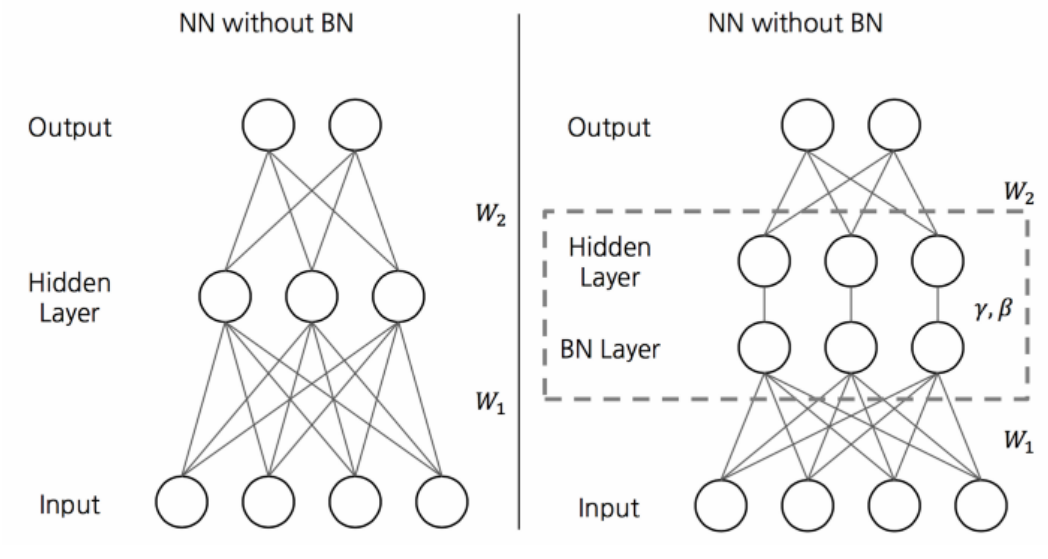
This approach is done by initializing the weights of your network to the ones of another network with the same architecture **pre-trained** on a large, generic dataset. **We often use this approach for computer vision projects.** It helps us a lot when our business issue doesn't have much data, but we can find another similar issue that does. In that case, we can use **transfer learning** to reduce overfitting.

Batch Normalization

Batch normalization (BN) is a technique to standardize the inputs to a network, applied to either the activations of a prior layer or inputs directly.

This approach allows each layer of a network to learn by itself a little bit more independently of other layers. BN is used to normalize the input layer by adjusting and scaling the activations.

Since BN has a regularizing effect it also means you can often remove dropout (which is helpful as dropout usually slows down training).



batch size *= the number of training examples in one forward/backward pass. The higher the batch size, the more memory space you'll need.*

Data augmentation.

Another way of dealing with overfitting is by improving the quality of your data. You might think of **outlier/noise** removal, however, in reality, their efficiency is quite limited. Another interesting way (especially in image-related tasks) is **data augmentation**. The goal is to randomly transform the training examples so that while they appear to the model to be different, they convey the same semantic information.

Competitive Learning in Neural Network

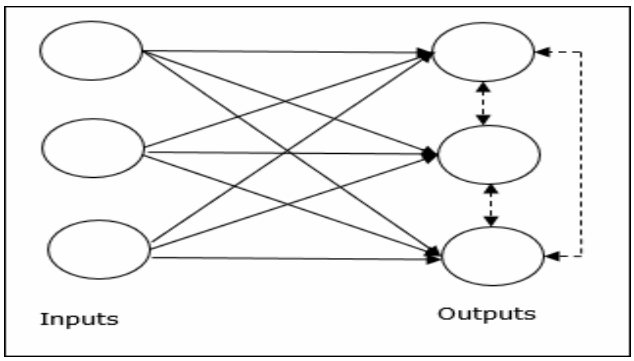
It is concerned with unsupervised training in which the output nodes try to compete with each other to represent the input pattern. To understand this learning rule we will have to understand competitive net which is explained as follows −

Winner-Takes-All Networks

These kinds of networks are based on the competitive learning rule and will use the strategy where it chooses the neuron with the greatest total inputs as a winner. The connections between the output neurons show the competition between them and one of them would be 'ON' which means it would be the winner and others would be 'OFF'.

Basic Concept of Competitive Network

This network is just like a single layer feed-forward network having feedback connection between the outputs. The connections between the outputs are inhibitory type, which is shown by dotted lines, which means the competitors never support themselves.



Basic Concept of Competitive Learning Rule

As said earlier, there would be competition among the output nodes so the main concept is - during training, the output unit that has the highest activation to a given input pattern, will be declared the winner. This rule is also called Winner-takes-all because only the winning neuron is updated and the rest of the neurons are left unchanged.

Mathematical formulation – Following are the three important factors for mathematical formulation of this learning rule –

- **Condition to be a winner** – Suppose if a neuron y_k wants to be the winner then there would be the following condition –

$$y_k = \begin{cases} 1 & if\, v_k > v_j\, for\, all\, j,\, j \neq k \\ 0 & otherwise \end{cases}$$

It means that if any neuron, say y_k , wants to win, then its induced local field $theoutputofsummationunit$, say v_k, must be the largest among all the other neurons in the network.

- **Condition of sum total of weight** – Another constraint over the competitive learning rule is, the sum total of weights to a particular output neuron is going to be 1. For example, if we consider neuron **k** then –

$$\sum_j w_{kj} = 1 \qquad for\, all\, k$$

- **Change of weight for winner** – If a neuron does not respond to the input pattern, then no learning takes place in that neuron. However, if a particular neuron wins, then the corresponding weights are adjusted as follows

$$\Delta w_{kj} = \begin{cases} -\alpha(x_j - w_{kj}), & if\, neuron\, k\, wins \\ 0, & if\, neuron\, k\, losses \end{cases}$$